

AI Usage DISCLOSURE FORM

1. Team Details

Team Name: Team Sudarshan

Project / Product Name: Streaming Live RAG (Samsung PRISM GenAI Hackathon, Theme 04)

Organization / Institution (if any): SRM Institute of Science and Technology, Kattankulathur

Submission Date: 25 September 2026

2. AI Usage Declaration

Did your team use any Artificial Intelligence (AI) in developing this project? Yes

The team used Claude (Anthropic), mainly through Claude Code, as a development assistant. Most code was AI-generated and then reviewed, tested, edited and tuned by the team. Separately, the product itself calls LLMs at runtime (OpenAI gpt-4.1 and gpt-5.4-mini) plus local open models (bge-small-en-v1.5, ms-marco-MiniLM-L-6, nli-deberta-v3-xsmall, MiniCheck-RoBERTa-Large); these are components of the system, not development aids.

3. Purpose of AI Usage (Brief Details)

Idea generation / brainstorming Yes. Claude used to brainstorm the streaming architecture (controller, decomposer, fusion, grounding) and trade-offs; final design choices made and validated by the team through benchmark runs.

Code generation or assistance Yes. Claude Code generated most of the backend (Python/FastAPI) and frontend (Next.js/TypeScript) code; the team reviewed, edited, integrated and tuned it.

UI / UX design Yes. Claude assisted with the operator console layout, trace components and theming; team reviewed and refined.

Content creation Yes. Claude assisted with README, architecture docs, decision log, benchmark report text, demo script and the presentation deck.

Data analysis Yes, limited. Claude assisted in summarising benchmark results from the evaluation traces; all figures come from the team's stored benchmark run.

Testing / debugging Yes. Claude Code assisted in writing unit tests and the evaluation harness and in debugging; tests and gates run by the team.

Other Runtime AI inside the product is disclosed in Section 2. Datasets: ASQA (Apache-2.0) and a synthetic enterprise corpus written for evaluation.

4. Feature Origin Classification (Please add/delete based on the number of the feature)

Feature 1. Feature Name: Streaming retrieval controller (early retrieval, cancellation)

Self-Generated/AI-Generated/Both: Both

Description: Rule-based controller that reads the transcript chunk by chunk and launches cancellable provisional searches before the speaker finishes. Tool: Claude (Anthropic), via Claude Code. Prompt approach: natural-language task specs describing the required behaviour, interfaces and constraints, followed by iterative fix/refine requests. Output: code and text drafts implementing the feature. Modification: the team reviewed, edited, tested and tuned the output against the benchmark gates.

Feature 2. Feature Name: Multi-intent decomposer

Self-Generated/AI-Generated/Both: Both

Description: Splits one utterance into up to 4 sub-queries with a merge guard and nearest-neighbour few-shot examples from ASQA's train split. Tool: Claude (Anthropic), via Claude Code. Prompt approach: natural-language task specs describing the required behaviour, interfaces and constraints, followed by iterative fix/refine requests. Output: code and text drafts implementing the feature. Modification: the team reviewed, edited, tested and tuned the output against the benchmark gates.

Feature 3. Feature Name: Hybrid retrieval and coverage-guaranteed fusion

Self-Generated/AI-Generated/Both: Both

Description: BM25 + dense search per sub-query, RRF, cross-encoder rerank, margin cut and a per-intent quota. Tool: Claude (Anthropic), via Claude Code. Prompt approach: natural-language task specs describing the required behaviour, interfaces and constraints, followed by iterative fix/refine requests. Output: code and text drafts implementing the feature. Modification: the team reviewed, edited, tested and tuned the output against the benchmark gates.

Feature 4. Feature Name: Grounded synthesis and citation validation

Self-Generated/AI-Generated/Both: Both

Description: Streams the answer one verified sentence at a time; citations built server-side; NLI + MiniCheck support checks; unsupported claims become uncertainty. Tool: Claude (Anthropic), via Claude Code. Prompt approach: natural-language task specs describing the required behaviour, interfaces and constraints, followed by iterative fix/refine requests. Output: code and text drafts implementing the feature. Modification: the team reviewed, edited, tested and tuned the output against the benchmark gates.

Feature 5. Feature Name: Claim-level refinement and suppression

Self-Generated/AI-Generated/Both: Both

Description: Late details refine the existing answer (KEEP/EDIT/ADD/DROP, delta search only); presentation-only turns trigger no retrieval. Tool: Claude (Anthropic), via Claude Code. Prompt approach: natural-language task specs describing the required behaviour, interfaces and constraints, followed by iterative fix/refine requests. Output: code and text drafts implementing the feature. Modification: the team reviewed, edited, tested and tuned the output against the benchmark gates.

Feature 6. Feature Name: Telemetry and observability

Self-Generated/AI-Generated/Both: Both

Description: 27-field per-turn trace, cost ledger, OpenTelemetry export and provisioned Grafana dashboard. Tool: Claude (Anthropic), via Claude Code. Prompt approach: natural-language task specs describing the required behaviour, interfaces and constraints, followed by iterative fix/refine requests. Output: code and text drafts implementing the feature. Modification: the team reviewed, edited, tested and tuned the output against the benchmark gates.

Feature 7. Feature Name: Evaluation suite

Self-Generated/AI-Generated/Both: Both

Description: Six acceptance gates, batch baseline, ablations, ASQA dataset build and edge-case analysis. Tool: Claude (Anthropic), via Claude Code. Prompt approach: natural-language task specs describing the required behaviour, interfaces and constraints, followed by iterative fix/refine requests. Output: code and text drafts implementing the feature. Modification: the team reviewed, edited, tested and tuned the output against the benchmark gates.

Feature 8. Feature Name: Operator console (frontend)

Self-Generated/AI-Generated/Both: Both

Description: Next.js console that visualises controller timeline, sub-queries, evidence, version diffs and metrics over AG-UI events. Tool: Claude (Anthropic), via Claude Code. Prompt approach: natural-language task specs describing the required behaviour, interfaces and constraints, followed by iterative fix/refine requests. Output: code and text drafts implementing the feature. Modification: the team reviewed, edited, tested and tuned the output against the benchmark gates.

Feature 9. Feature Name: Documentation and submission deck

Self-Generated/AI-Generated/Both: Both

Description: README, architecture brief, decision log, benchmark report, demo script and PPT. Tool: Claude (Anthropic). Prompt approach: requests to draft or restructure documents from the team's code, benchmark results and template. Output: draft text and

slides. Modification: the team checked claims against the stored benchmark run and corrected unsupported figures.

5. Ethical & Compliance Confirmation

AI usage complies with guidelines and policies. **Yes**

No proprietary or copyrighted data misused. **I Agree**

6. Declaration & Sign-Off

Name of Team Representative: Rachit Gandhi

Role: Team Lead

Signature: 

Date: 25 September 2026